use in your applications: JVM, loggers, Spring MVC, Spring WebFlux, RestTemplate, WebClient, data sources, Hibernate, Spring Data, RabbitMQ, and more.

When Spring Cloud Gateway is in the classpath, as in the case of Edge Service, additional metrics are exported regarding the gateway routes. Some libraries, like Resilience4J, contribute dedicated Micrometer instrumentation through specific dependencies to register additional metrics.

Open the build.gradle file in the Edge Service project (edge-service), and add the following dependency to include Micrometer instrumentation for Resilience4J. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 13.15 Adding dependency for Micrometer Resilience4J

```
dependencies {
  ...
  runtimeOnly 'io.github.resilience4j:resilience4j-micrometer'
}
```

Now that we've configured Spring Boot to expose metrics, let's see how we can configure Prometheus to scrape them and Grafana to visualize them.

13.3.2 Monitoring metrics with Prometheus and Grafana

Like Loki, Prometheus collects and stores metrics. It even provides a GUI to visualize them and to define alarms, but we'll use Grafana for that since it's a more comprehensive tool.

Metrics are stored as time-series data, containing the timestamp when they were registered and, optionally, labels. In Prometheus, labels are key/value